



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

«Визуализация процесса водной эрозии рельефа»

Студент

ИУ7-52Б

(группа)

(подпись, дата)

В. М. Пантелеев

(И.О. Фамилия)

Руководитель курсовой
работы

(подпись, дата)

З. Н. Русакова

(И.О. Фамилия)

2024 г.

РЕФЕРАТ

Курсовая работа 44 с., 14 рис., 1 табл., 16 ист., 1 прил.

**КОМПЬЮТЕРНАЯ ГРАФИКА, Z-БУФЕР, ЗАКРАСКА, ОСВЕЩЕНИЕ,
СЦЕНА, РЕЛЬЕФ, ЛАНДШАФТ, ЭРОЗИЯ.**

Целью данной работы является создание программного обеспечения для визуализации процесса водной эрозии рельефа с возможностью изменения физических и визуальных параметров.

Проведено исследование зависимости времени построения изображения ландшафта от размера карты высот. Из результатов исследования следует, что время отрисовки сцены увеличивается линейно, при увеличении размерности высотной карты.

СОДЕРЖАНИЕ

РЕФЕРАТ	3
СОДЕРЖАНИЕ	4
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	6
ВВЕДЕНИЕ	7
1 Аналитическая часть	8
1.1 Описание объектов сцены	8
1.2 Формализация задачи	8
1.2.1 Рельеф местности	9
1.2.2 Вода (визуализация эрозии)	9
1.2.3 Источник света	9
1.2.4 Камера	9
1.3 Способы представления моделей	11
1.3.1 Каркасные модели	11
1.3.2 Поверхностные модели	11
1.3.3 Объёмные модели	11
1.3.4 Выбор метода представления моделей	12
1.4 Анализ методов удаления невидимых линий и поверхностей	13
1.4.1 Алгоритм Варнока	13
1.4.2 Алгоритм, использующий Z буфер	14
1.4.3 Прямая и обратная трассировка лучей	14
1.4.4 Выбор метода удаления невидимых линий и поверхностей	15
1.5 Анализ методов закраски	17
1.5.1 Однотонная закраска	17
1.5.2 Метод Гуро	18
1.5.3 Метод Фонга	18
1.5.4 Выбор метода закраски	19

1.6	Анализ методов визуализации водной эрозии рельефа	19
1.6.1	Карта воды	19
1.6.2	Капельный метод	20
1.6.3	Выбор метода эрозии	20
2	Конструкторская часть	21
2.1	Требования к программному обеспечению	21
2.2	Описание структур данных	21
2.3	Общий алгоритм синтеза изображения	22
2.4	Алгоритм Z-буфера	24
2.5	Алгоритм процесса эрозии на основе высотных карт	25
3	Технологическая часть	28
3.1	Средства реализации	28
3.2	Структура программного обеспечения	28
3.3	Реализация алгоритмов	29
3.3.1	Отрисовка треугольника	29
3.3.2	Водная эрозия	31
3.4	Интерфейс программного обеспечения	33
4	Исследовательская часть	38
4.1	Технические характеристики ЭВМ	38
4.2	Результаты исследования	38
	ЗАКЛЮЧЕНИЕ	41
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
	ПРИЛОЖЕНИЕ А	44

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В представленной расчетно-пояснительной записке используются следующие сокращения и обозначения:

- 1) ПО – программное обеспечение;
- 2) RGB – Red, Green, Blue;
- 3) GUI – Graphical User Interface;
- 4) ЭВМ – электронно-вычислительная машина.

ВВЕДЕНИЕ

Цель работы — создание программного обеспечения для визуализации процесса водной эрозии рельефа с возможностью изменения физических и визуальных параметров.

Для достижения поставленной цели требуется решить следующие задачи:

- проанализировать алгоритмы компьютерной графики для построения изображения;
- выбрать подходящие алгоритмы для построения изображения рельефа и визуализации водной эрозии;
- спроектировать программное обеспечение;
- выбрать средства реализации;
- реализовать ПО для построения изображения рельефа и визуализации водной эрозии;
- исследовать зависимости времени построения изображения от размера карты высот;
- обосновать полученные результаты.

1 Аналитическая часть

В данной части формализация объектов сцены, а также анализ существующих алгоритмов построения изображений и выбор наиболее подходящих из них для выполнения поставленной задачи.

1.1 Описание объектов сцены

Сцена состоит из следующих объектов:

1. **рельеф местности** — представляет собой трёхмерную поверхность, описывающую топографию земли;
2. **вода** — визуализация эрозии;
3. **точечный источник света** — представляет собой материальную точку;
4. **камера** — определяет точку зрения наблюдателя в сцене.

1.2 Формализация задачи

Входными данными программного обеспечения являются характеристики источника света, информация о положении камеры и параметры эрозии почвы. Выходными данными является визуализация водной эрозии рельефа.

Формализация задачи в виде IDEF0 диаграммы приведена на рисунке 1.1.

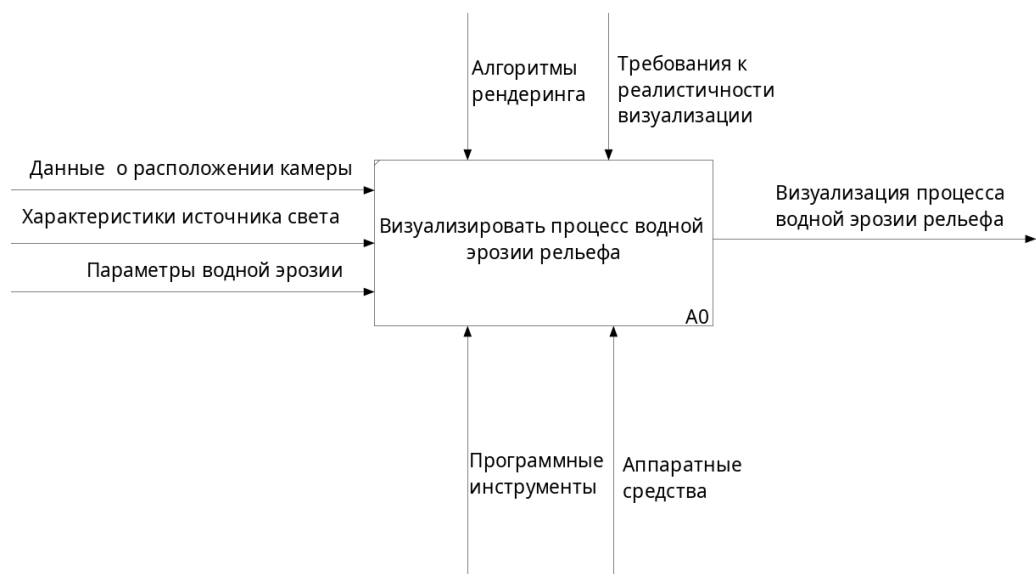


Рисунок 1.1 — Формализация задачи в виде IDEF0 диаграммы нулевого уровня

1.2.1 Рельеф местности

Рельеф местности является ключевым объектом сцены, определяющим основную геометрию и визуальное представление поверхности земли. Он задаётся высотной картой, представляющей собой двумерный массив, где каждой точке (x, y) соответствует высота z .

Для отображения рельефа используется полигональная сетка, состоящая из вершин и граней. Каждая вершина имеет координаты в трёхмерном пространстве (x, y, z) . Грани в виде треугольников образуются путём соединения вершин.

1.2.2 Вода (визуализация эрозии)

Вода перемещается по поверхности рельефа, взаимодействуя с ним и изменяя его форму путём смыва и переноса частиц грунта. Это приводит к формированию русел, размыванию склонов и другим эффектам эрозии.

Вода представляется в виде множества капель, скользящих по поверхности. При движении воды по рельефу учитываются физические параметры, такие как, наклон поверхности. Сами капли визуального представления не имеют, а только оказывают влияние на рельеф.

1.2.3 Источник света

Основные характеристики источника света:

- **интенсивность** — определяет яркость света и влияет на освещённость поверхности;
- **положение** — задаёт координаты источника света в пространстве;
- **цвет** — описывается компонентами RGB и влияет на оттенок освещения.

1.2.4 Камера

Камера определяет точку обзора в сцене и отвечает за проекцию трёхмерного рельефа на двумерный экран. Параметры камеры включают:

- **положение** — координаты камеры в трёхмерном пространстве (x, y, z) ;
- **направление взгляда** — вектор, определяющий, куда направлена ка-

мера;

— **угол обзора** — определяет поле зрения камеры и влияет на перспективу;

— **параметры проекции** — параметры ближней и дальней плоскостей отсечения.

1.3 Способы представления моделей

В компьютерной графике существует несколько способов представления трёхмерных моделей, каждый из которых имеет свои преимущества и недостатки. Выбор подходящего метода зависит от требований к визуализации, производительности и объёму данных.

1.3.1 Каркасные модели

Простейший вид моделей, который содержит информацию только о вершинах и ребрах объектов. Это моделирование самого низкого уровня, которое имеет ряд серьезных ограничений. Большинство ограничений возникает из-за недостатка информации о гранях, которые заключены между ребрами. Невозможно выделить внутреннюю и внешнюю область изображения твердого объемного тела. Проблемой этой модели заключается в том, что она не позволяет отличить видимые грани от невидимых [1].

1.3.2 Поверхностные модели

Поверхностные модели описывают объект с помощью граней, ограничивающих его форму. Эти модели позволяют разделить внутреннюю и внешнюю области тела, предоставляя больше возможностей для анализа и визуализации, чем каркасные.

Поверхностное представление может быть выполнено с использованием многоугольников, чаще всего треугольников, что делает такие модели удобными для применения текстур и освещения. Они позволяют визуализировать объект более реалистично, а также обнаруживать пересечения или коллизии между поверхностями.

Такие модели имеют свои ограничения. Например, они не содержат информации о внутренней структуре объекта. Кроме того, вычислительная сложность увеличивается с ростом детализации объекта [1].

1.3.3 Объёмные модели

Объёмные модели описывают объект с учётом его внутренней структуры, предоставляя наиболее полное представление о форме и содержимом. Для

создания таких моделей используются различные подходы, включая воксели (трёхмерные пиксели) и неявные поверхности, задаваемые математическими функциями [1].

1.3.4 Выбор метода представления моделей

Для задачи визуализации процесса водной эрозии рельефа необходимо обеспечить отображение изменений поверхности при сохранении высокой производительности. Исходя из этого, наиболее подходящими являются поверхностные модели. Они позволяют эффективно хранить и визуализировать сложный рельеф с высокой детализацией, необходимой для отображения процессов эрозии.

Недостатки других методов делают их менее пригодными для данной задачи. Объёмные модели избыточны для визуализации поверхности рельефа, а простые каркасные модели недостаточны для демонстрации процесса эрозии.

1.4 Анализ методов удаления невидимых линий и поверхностей

Алгоритмы удаления невидимых линий или поверхностей можно классифицировать по способу выбора системы координат или пространства, в котором они работают. Алгоритмы, работающие в объектном пространстве, имеют дело с физической системой координат, в которой описаны эти объекты. При этом получаются весьма точные результаты, ограниченные, вообще говоря, лишь точностью вычислений. Полученные изображения можно свободно увеличивать во много раз. Алгоритмы, работающие в пространстве изображения, особенно полезны в тех приложениях, где необходима высокая точность.

Алгоритмы же, работающие в пространстве изображения, имеют дело с системой координат того экрана, на котором объекты визуализируются. При этом точность вычислений ограничена разрешающей способностью экрана [1].

В данном разделе рассмотрены алгоритм Варнока, Z-буфер, а также методы прямой и обратной трассировки лучей.

1.4.1 Алгоритм Варнока

Данный алгоритм работает в пространстве изображения. В пространстве изображения рассматривается окно и решается вопрос о том, пусто ли оно, или его содержимое достаточно просто для визуализации. Если это не так, то окно разбивается на фрагменты до тех пор, пока содержимое фрагмента не станет достаточно простым для визуализации или его размер не достигнет требуемого предела разрешения [2].

Пример разбиения окна показан на рисунке 1.2.

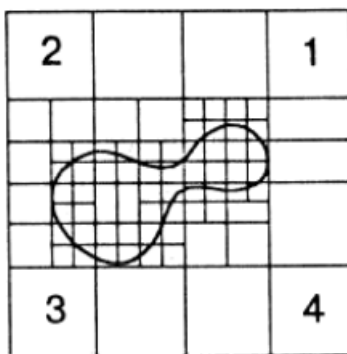


Рисунок 1.2 — Пример разбиения окна в алгоритме Варнока

1.4.2 Алгоритм, использующий Z буфер

Одним из самых простых алгоритмов удаления невидимых граней и поверхностей является метод Z-буфера (буфера глубины). Алгоритм работает в пространстве изображения. В силу крайней простоты этого метода часто встречаются его аппаратные реализации [3].

Размеры Z-буфера равны размерам окна, таким образом, каждому пикселю окна соответствует ячейка Z-буфера. В этой ячейке хранится значение глубины пикселя. Перед растеризацией сцены Z-буфер заполняется значением, соответствующим максимальной глубине. В случае, когда глубина характеризуется значением w , максимальной глубине соответствует нулевое значение.

Анализ видимости происходит при растеризации граней, для каждого пикселя рассчитывается глубина и сравнивается со значением в Z-буфере, если рисуемый пиксель ближе (его w больше значения в Z-буфере), то пиксель рисуется, а значение в Z-буфере заменяется его глубиной. Если пиксель дальше, то пиксель не рисуется и Z-буфер не изменяется, текущий пиксель дальше того, что нарисован ранее, а значит, невидим [2].

Пример организации Z-буфера показан на рисунке 1.3.

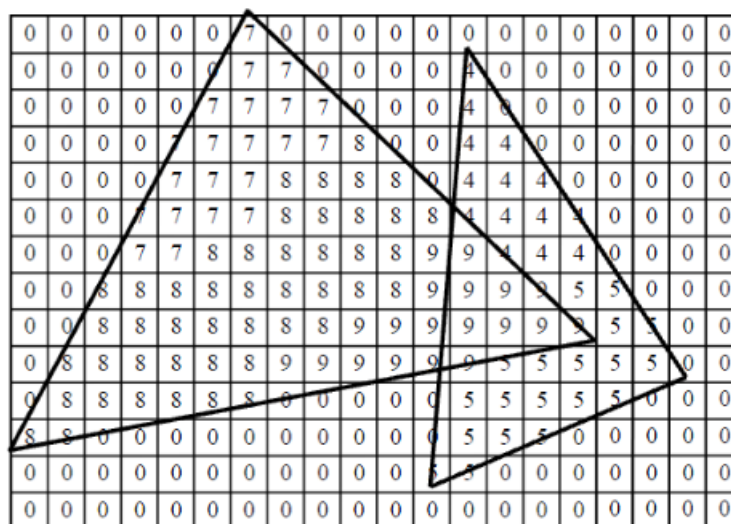


Рисунок 1.3 — Пример организации Z-буфера

1.4.3 Прямая и обратная трассировка лучей

В этом алгоритме для удаления скрытых линий и поверхностей можно использовать тот факт, что в реальной природе источник света испускает луч

света, который, «путешествуя» по пространству, в конечном счете «натыкается» на какую-либо преграду, которая прерывает распространение этого светового луча. В какой-либо точке пути с лучом света может случиться любая комбинация трех вещей: поглощение, отражение (рефлексия) и преломление (рефракция) [2].

Прямая трассировка лучей

Суммарная интенсивность светового луча, которая была «потеряна» вследствие поглощения, преломления и отражения, должна быть в точности равной исходящей (начальной) интенсивности этого луча. Далее отраженные и/или преломленные лучи достигают других поверхностей, где их поглощающие, отражающие и преломляющие способности снова вычисляются, основываясь на результатах вычислений входящих лучей. Таким образом, луч, пропущенный через пиксель картиной плоскости, образует сложный разделяющийся путь [2].

Обратная трассировка лучей

Если проследить за лучами света, выпущенным источником света, то можно убедиться, что весьма немногие дойдут до наблюдателя. Следовательно, этот процесс вычислительно неэффективен. Поэтому было предложено отслеживать (трассировать) лучи в обратном направлении, т.е. от наблюдателя к объекту и далее к источнику освещения. При этом, интенсивность в пикселе, через который проходит луч, рассчитывалась бы интегрально с учетом всех слагаемых: пересечения, отражения, преломления луча с разными объектами и характеристик источников света. Такой подход получил название метод обратной трассировки лучей [2].

1.4.4 Выбор метода удаления невидимых линий и поверхностей

Z-буфер является подходящим решением для данной задачи, так как он обрабатывает каждую грань и каждый пиксел независимо, обеспечивая линейную зависимость времени выполнения от количества объектов в сцене. Это позволяет программе эффективно справляться с изменениями геометрии рельефа, которые происходят в реальном времени.

В отличие от алгоритмов, таких как прямая и обратная трассировка лучей, которые требуют сложных геометрических вычислений или предварительной сортировки полигонов, Z-буфер позволяет обновлять изображение сцены без значительных вычислительных затрат.

1.5 Анализ методов закраски

Световая энергия, падающая на поверхность от источника света, может быть поглощена, отражена и пропущена. Свойства отраженного света зависят от формы и направления источника света, а также от ориентации освещаемой поверхности и ее свойств. Свет, отраженный от объекта, может быть диффузным и зеркальным: диффузно отраженный свет рассеивается равномерно по всем направлениям, зеркальное отражение происходит от внешней поверхности объекта [3].

Диффузное отражение присуще матовым поверхностям. Матовой можно считать такую поверхность, размер шероховатостей которой уже настолько велик, что падающий луч рассеивается равномерно во все стороны. Такой тип отражения характерен, например, для гипса, песка, бумаги. Диффузное отражение описывается законом Ламберта, согласно которому интенсивность отраженного света пропорциональна косинусу угла между направлением на точечный источник света и нормалью к поверхности 1.1.

$$I_d = I \cdot K_d \cdot \cos \theta, \quad (1.1)$$

где I — интенсивность источника света, K_d — коэффициент, который учитывает свойства материала поверхности [4].

В задаче эрозии рельефа все поверхности являются матовыми. Поэтому в данном случае достаточно использовать диффузное отражение.

Существует три основных способа закраски объектов, заданных полигональными сетками. В порядке возрастания сложности ими являются [2]:

- 1) однотонная закраска;
- 2) метод Гуро (основан на интерполяции значений интенсивности);
- 3) метод Фонга (основан на интерполяции векторов нормали).

1.5.1 Однотонная закраска

Пусть $\vec{L}$ — вектор, направленный от центра поверхности на точечный источник света, $\vec{N}$ — нормаль к поверхности, $\vec{V}$ — вектор, направленный от центра поверхности к точке зрения (камере).

При однотонной закраске вычисляют один уровень интенсивности, кото-

рый используется для закрашки всего многоугольника. При этом предполагается, что:

- 1) источник света расположен в бесконечности, поэтому произведение $\vec{L} \cdot \vec{N}$ постоянно на всей полигональной грани;
- 2) наблюдатель находится в бесконечности, поэтому произведение $\vec{N} \cdot \vec{V}$ постоянно на всей полигональной грани;
- 3) многоугольник представляет реальную моделируемую поверхность, а не является аппроксимацией криволинейной поверхности.

Если какое-либо из первых двух предположений оказывается неприемлемым, можно воспользоваться усредненными значениями $\vec{L}$ и $\vec{V}$, вычисленными, например, в центре многоугольника [2].

1.5.2 Метод Гуро

Этот метод позволяет устранить дискретность изменения интенсивности. Процесс закрашки по методу Гуро осуществляется в четыре этапа [2]:

- 1) вычисляются нормали ко всем полигона;
- 2) определяются нормали в вершинах путем усреднения нормалей по всем полигональным граням, которым принадлежит вершин;
- 3) используя нормали в вершинах и применяя произвольный метод закрашки, вычисляются значения интенсивности в вершинах;
- 4) каждый многоугольник закрашивается путем линейной интерполяции значений интенсивностей в вершинах сначала вдоль каждого ребра, а затем и между ребрами вдоль каждой сканирующей строки.

1.5.3 Метод Фонга

В методе закрашки, разработанном Фонгом, используется интерполяция вектора нормали N к поверхности вдоль видимого интервала на сканирующей строке внутри многоугольника, а не интерполяция интенсивности. Интерполяция выполняется между начальной и конечной нормальями, которые сами тоже являются результатами интерполяции вдоль ребер многоугольника между нормальями в вершинах. Нормали в вершинах, в свою очередь, вычисляются так же, как в методе закрашки, построенном на основе интерполяции интенсивности [2].

1.5.4 Выбор метода закраски

Для реализации программы по визуализации процесса водной эрозии рельефа был выбран метод закраски по Гуро. Закраска по Гуро позволяет получить плавные переходы цвета между вершинами полигонов, что достаточно для визуального восприятия рельефа как непрерывной поверхности.

1.6 Анализ методов визуализации водной эрозии рельефа

Процесс водной эрозии почв состоит из разрушения почв каплями дождя и потоками воды, сформировавшиеся на поверхности склона в результате выпадения дождя (природных, искусственных) или таяния снега, транспортировке почвенных частиц и агрегатов и их отложения [5].

Существует несколько подходов к симуляции эрозии, каждый из которых имеет свои особенности. В данном разделе рассматривается два основных метода.

1.6.1 Карта воды

В данном подходе ландшафт (высотная карта) дополняется двумя дополнительными картами (или слоями):

- карта воды — хранит, сколько воды находится над каждой ячейкой высотной карты;
- карта осадка — хранит, сколько грунта (осадка) переносит или содержит вода в каждой ячейке.

Итерационно выполняются следующие стадии:

- 1) добавление воды (дождь) — во все или выбранные ячейки «льётся» некоторое количество воды;
- 2) сток воды — каждая ячейка пытается «передать» воду в соседние ячейки, которые ниже по высоте (с учётом уже имеющейся в них воды);
- 3) эрозия и перенос осадка — при стоке вода «смывает» часть грунта и добавляет его в карту осадка; если скорость потока или уклон недостаточны, осадок может выпасть обратно на поверхность;
- 4) испарение — часть воды испаряется из каждой ячейки, оставляя отложенный осадок в виде изменения высот.

1.6.2 Капельный метод

Вместо заполненной всей карты водой, здесь эрозия представляется в виде отдельных капель, которые:

- 1) создаются в некоторой точке ландшафта с начальным объёмом воды;
- 2) скользят по рельефу, каждый раз перемещаясь к соседу с более низкой высотой, постепенно смывая грунт;
- 3) в капле хранится «количество осадка», которое может быть отложено, если капля достигла локальной ямки;
- 4) капля испаряется или прекращает движение, когда вода в ней заканчивается или когда уклон исчезает.

Процесс повторяется для большого числа капель в разных точках карты.

1.6.3 Выбор метода эрозии

Для решения задачи формирования реалистичных русел, оврагов и направленного переноса грунта наиболее предпочтительным оказывается «капельный» метод, поскольку он позволяет моделировать локальные потоки воды и точечный процесс переноса осадка, тогда как «глобальная» карта воды приводит к избыточно равномерному размягчению рельефа и недостаточно ярко выраженным формам.

Вывод

В ходе аналитической части было принято решение представить рельеф в виде полигональной поверхностной модели. Для отсечения невидимых граней будет использован алгоритм Z-буфера, для закрашки выбран метод Гуро, а для эрозии – капельный метод

2 Конструкторская часть

В данном разделе будут рассмотрены требования к программе и алгоритмы визуализации сцены и процесса водной эрозии.

2.1 Требования к программному обеспечению

Программа должна предоставлять следующие возможности:

Динамическая визуализация процесса водной эрозии рельефа:

- исходная топография генерируется случайным образом;
- визуализация отображает изменения рельефа в реальном времени.

Настройка физических параметров эрозии:

- возможность задания числа и шага итераций эрозии
- возможность изменения силы осаждения наносов и силы эрозии;
- возможность изменения числа капель.

Реализация источника света:

- один источник света в сцене;
- возможность изменения интенсивности и цвета источника света.

Графический интерфейс пользователя (GUI):

- отображение результатов симуляции в реальном времени;
- инструменты для настройки параметров эрозии и освещения;
- возможность перезапуска визуализации.

2.2 Описание структур данных

Для формирования общего алгоритма синтеза изображения в данной программе, необходимо ввести определения использующихся в ней структур данных:

Сцена:

- полигональная модель рельефа;
- модель эрозии;
- камера;
- источник освещения.

Модель рельефа:

- массив вершин;

- массив полигонов;
- массив векторов нормалей.

Камера:

- координаты положения в пространстве;
- координаты точки наблюдения;
- угол обзора
- ближняя и дальняя плоскости отображения;
- вектор ориентации.

Источник освещения:

- координаты положения в пространстве;
- цвет;
- интенсивность.

Модель эрозии:

- число итераций эрозии;
- шаг итераций;
- сила эрозии;
- сила отложения остатков;
- количество капель.

2.3 Общий алгоритм синтеза изображения

Визуализация сцены происходит поэтапно. Описание алгоритма синтеза изображения представлено на рисунке 2.1. На вход подаются параметры камеры и источника освещения. На выходе – синтезированное изображение.

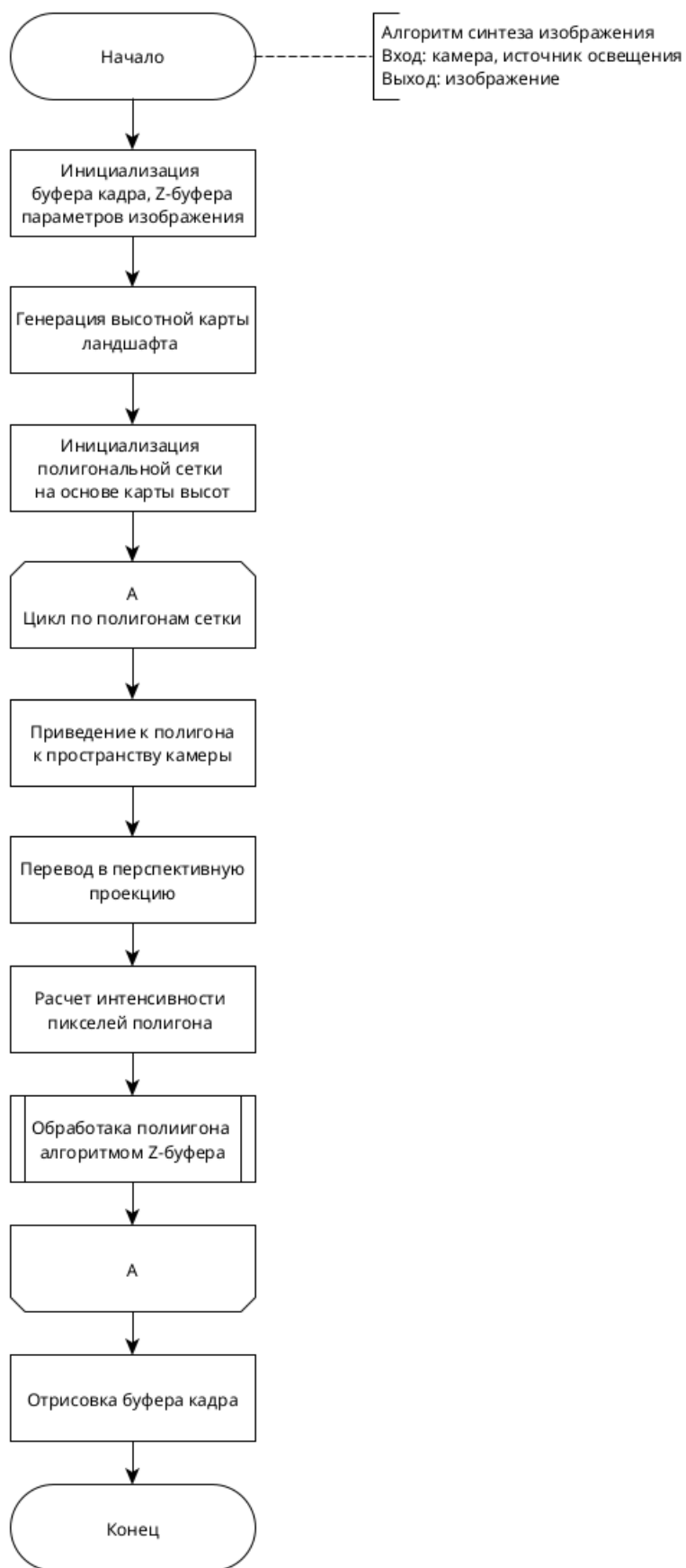


Рисунок 2.1 — Описание алгоритма синтеза изображения

2.4 Алгоритм Z-буфера

Алгоритм Z-буфера используется для удаления невидимых поверхностей в процессе рендеринга сцены. Он обеспечивает корректное отображение полигонов, скрывая те, которые находятся за другими объектами с точки зрения наблюдателя. Описание алгоритма представлено на рисунке 2.2.

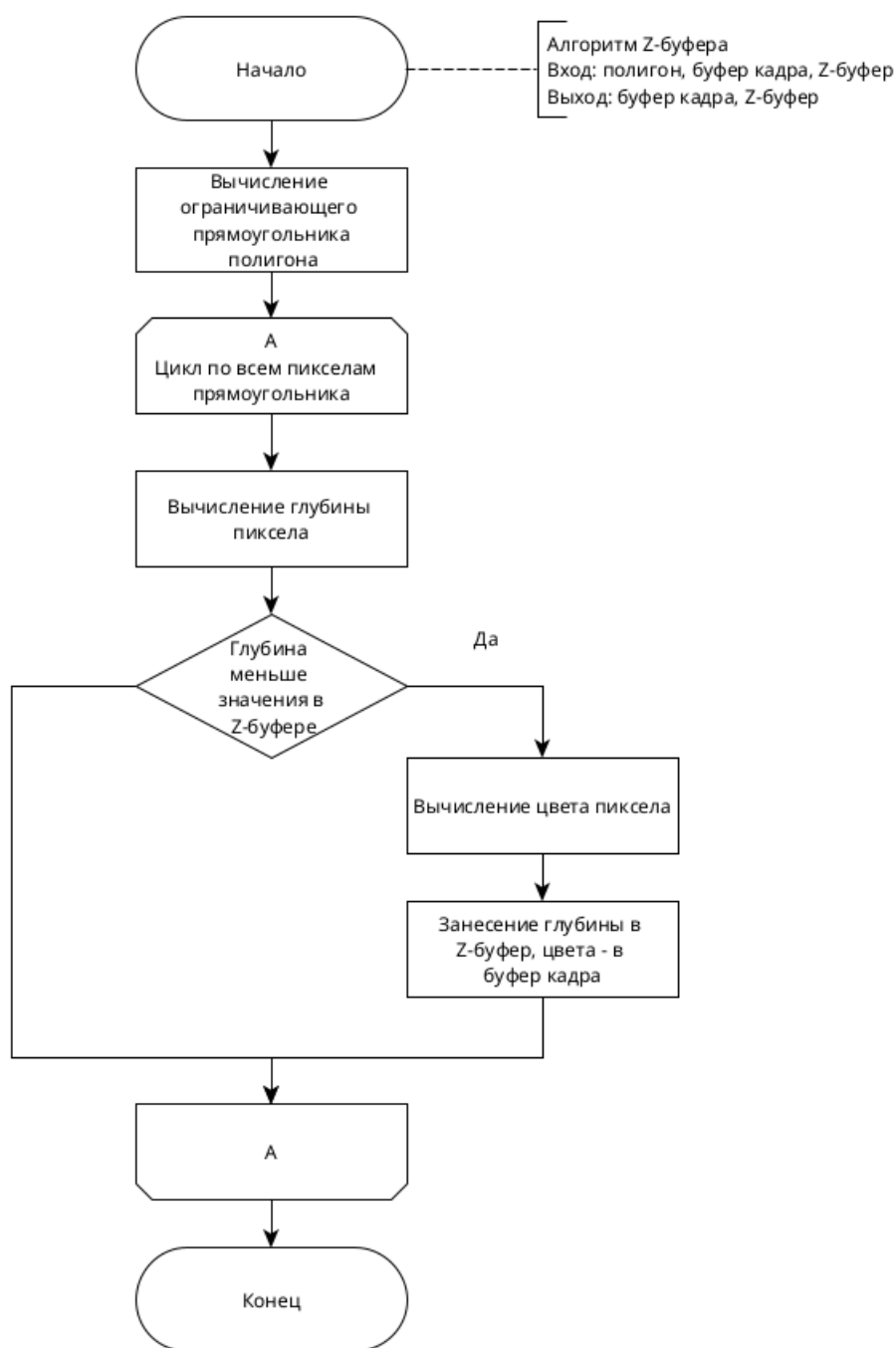


Рисунок 2.2 — Описание алгоритма Z-буфера

2.5 Алгоритм процесса эрозии на основе высотных карт

Алгоритм гидравлической эрозии на основе высотных карт моделирует процесс размывания рельефа водой. Он обновляет высотную карту рельефа в соответствии с физическими параметрами эрозии. Описание алгоритма приведено на рисунках 2.3 – 2.4.

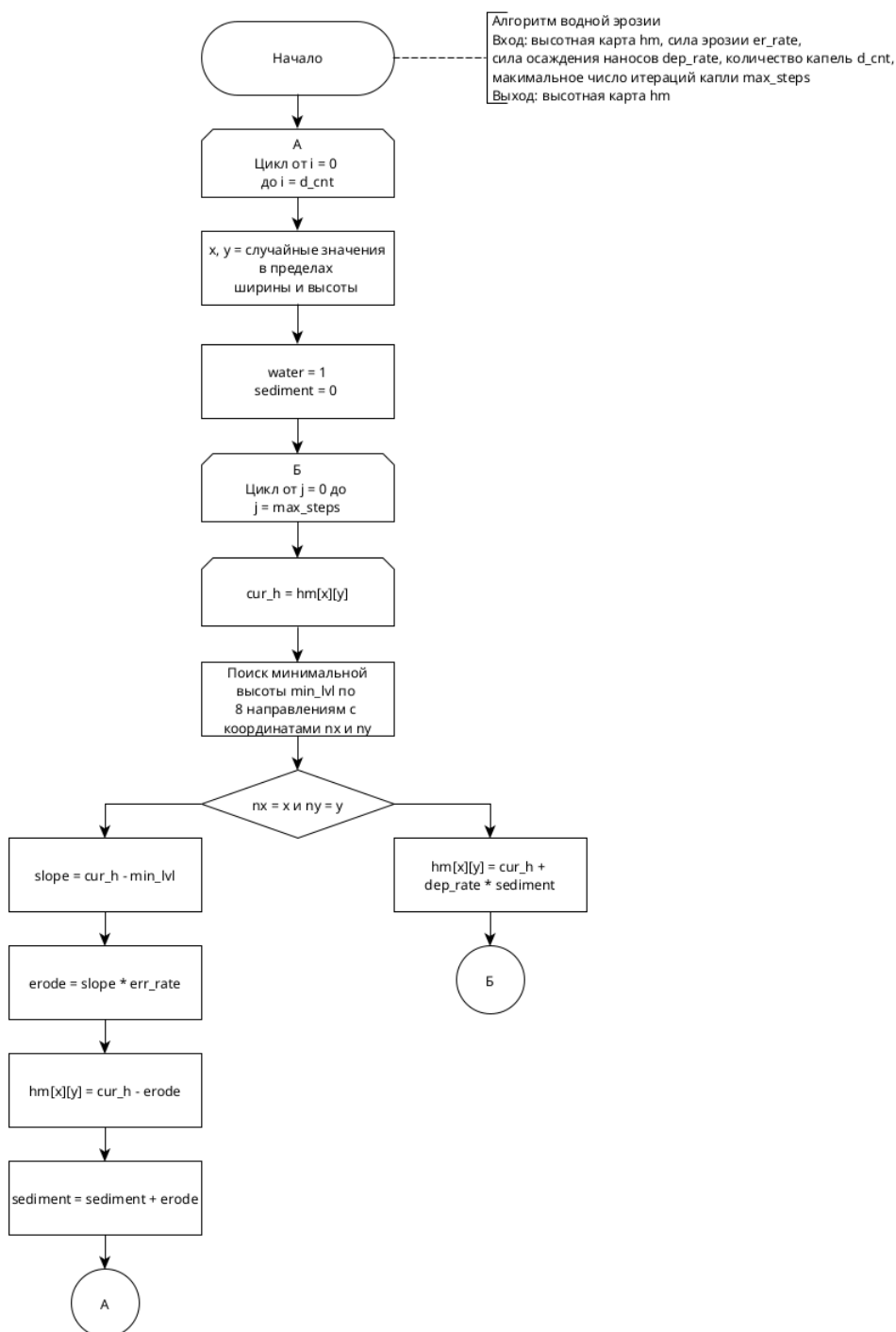


Рисунок 2.3 — Описание алгоритма водной эрозии

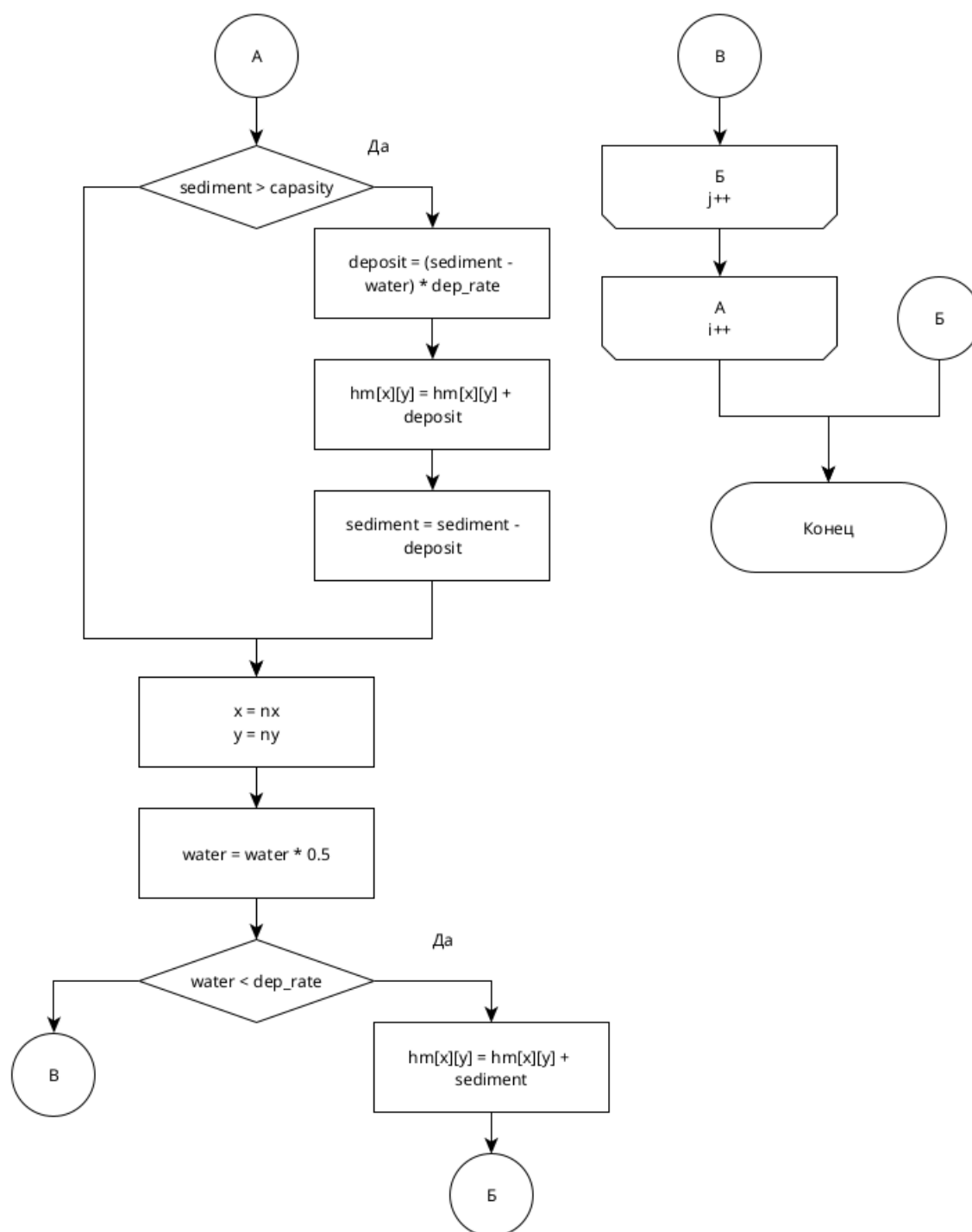


Рисунок 2.4 — Описание алгоритма водной эрозии (продолжение)

Капля перемещается от высоких уровней к низким, забирая часть грунта. Если у капли нет направлений для перемещения (впадина или плато) или произошло ее испарение, часть переносимого грунта осаждается, размывая почву.

Вывод

Были описаны требования к программному обеспечению, структуры данных и алгоритмы для построения сцены в пространстве изображения.

3 Технологическая часть

В данной части рассматривается выбор средств реализации, описывается структура классов программы и приводится интерфейс программного обеспечения.

3.1 Средства реализации

В данной работе для реализации был выбран язык программирования C++ [6]. Выбор обусловлен тем, что средствами языка можно реализовать все алгоритмы структуры данных, выбранные в результате проектирования.

В качестве среды разработки был выбран CLion [7] от компании JetBrains [8]. Эта среда обеспечивает удобство работы с проектами на C++, интеграцию с системой контроля версий, а также поддержку инструментария для отладки и профилирования кода.

Для разработки графического интерфейса был выбран QtCreator [9]. Для отображения сцены использована библиотека Qt [10]. Для реализации векторных вычислений, таких как операции над точками, векторами и матрицами, используется библиотека GLM [11] (OpenGL Mathematics).

Для управления структурами данных, включая полигональные сетки рельефа, высотные карты, карты воды и отложений, используются стандартные средства языка C++ и объектно-ориентированный подход.

3.2 Структура программного обеспечения

Для реализации программного обеспечения была выбрана объектно–ориентированная парадигма программирования. В связи с этим были реализованы следующие классы:

- 1) Camera – класс, задающий камеру;
- 2) Light – класс, задающий источник света;
- 3) Renderer – Класс, отвечающий за визуализацию полигональной сетки;
- 4) Mesh – класс, реализующий полигональную сетку;
- 5) Erosion – класс, отвечающий за визуализацию водной эрозии;
- 6) RenderWidget – класс, содержащий генератор полигональной сетки, камеру, источник освещения и визуализатор эрозии.

Структура программы в виде диаграммы классов изображена на рисунке 3.1.

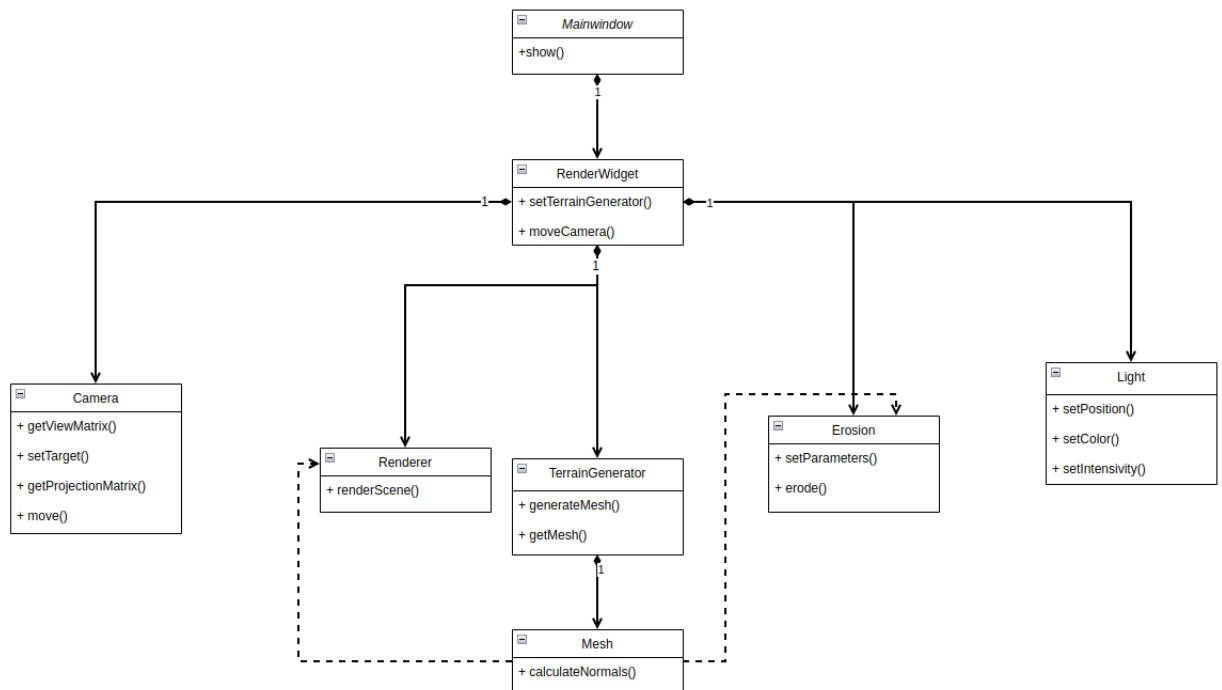


Рисунок 3.1 — Диаграмма классов

3.3 Реализация алгоритмов

3.3.1 Отрисовка треугольника

На листинге 3.1 приведена реализация алгоритма отрисовки треугольника с использованием Z буфера.

Листинг 3.1 — Реализация функции renderTriangle

```

void Renderer::drawTriangleWithLocks(const Vertex& v0, const Vertex& v1,
    const Vertex& v2,
                                     const Light& light, const glm::mat4&
                                     mvpMatrix)
{
    for (int y = (int)minY; y <= (int)maxY; y++)
    {
        for (int x = (int)minX; x <= (int)maxX; x++)
        {
            glm::vec3 p((float)x + 0.5f, (float)y + 0.5f, 0.0f);
            float wA = edgeFunction(p1, p2, p);
            float wB = edgeFunction(p2, p0, p);

```

```

float wC = edgeFunction(p0, p1, p);

if (area > 0 && wA > 0 && wB > 0 && wC > 0 || area <= 0 && wA <=
    0 && wB <= 0 && wC <= 0)
{
    float lambda0 = wA / area;
    float lambda1 = wB / area;
    float lambda2 = wC / area;

    float invW = (lambda0 / w0) + (lambda1 / w1) + (lambda2 / w2
        );

    glm::vec3 fragPos = (pos0_over_w * lambda0 + pos1_over_w *
        lambda1 + pos2_over_w * lambda2) / invW;
    glm::vec3 fragNormal = (norm0_over_w * lambda0 +
        norm1_over_w * lambda1 + norm2_over_w * lambda2) /
        invW;
    fragNormal = glm::normalize(fragNormal);

    float z = (p0.z * lambda0 + p1.z * lambda1 + p2.z * lambda2)
        ;

    int index = x + y * m_width;

    zBufferMutex.lock();
    float currentDepth = m_zBuffer[index];
    if (z < currentDepth)
    {
        m_zBuffer[index] = z;
        zBufferMutex.unlock();

        glm::vec3 toLight = glm::normalize(lightPos - fragPos);
        float lambert = glm::max(glm::dot(fragNormal, toLight),
            0.0f);
        glm::vec3 worldPos = lambda0 * v0.position + lambda1 *
            v1.position + lambda2 * v2.position;

        glm::vec3 colorByHeight = getColorByHeight(-worldPos.y);
        glm::vec3 fragColor = colorByHeight * lightColor *
            lambert;

        framebufferMutex.lock();
        m_frameBuffer.setPixelColor(x, y, QColor(
            static_cast<int>(
                fragColor.r * 255),
            static_cast<int>(

```

```
        fragColor.g * 255),  
        static_cast<int>(  
            fragColor.b * 255)  
    ));  
  
    framebufferMutex.unlock();  
}  
else  
    zBufferMutex.unlock();  
}  
}  
}
```

3.3.2 Водная эрозия

В листинге 3.2 представлена реализация алгоритма водной эрозии.

Листинг 3.2 — Реализация функции водной эрозии

```
void Erosion::applySingleIteration(TerrainGenerator& terrain)
{
    void Erosion::simulateDroplet(TerrainGenerator& terrain,
                                   int startX, int startY,
                                   int maxSteps,
                                   float initialWater)
    {
        int x = startX;
        int y = startY;

        float water = initialWater;
        float sediment = 0.0f;

        for (int step = 0; step < maxSteps; ++step)
        {
            float currentHeight = terrain.at(x, y);

            float lowestHeight = currentHeight;
            int nx = x;
            int ny = y;

            for (int dy = -1; dy <= 1; ++dy)
            {
                for (int dx_ = -1; dx_ <= 1; ++dx_)
                {
                    if (dx_ == 0 && dy == 0) continue;
                    int xx = x + dx_;
```

```

        int yy = y + dy;
        if (!inRange(xx, yy)) continue;

        float neighH = terrain.at(xx, yy);
        if (neighH < lowestHeight) {
            lowestHeight = neighH;
            nx = xx;
            ny = yy;
        }
    }

    if (nx == x && ny == y)
    {

        float deposit = sediment * m_depositionRate;
        terrain.setAt(x, y, currentHeight + deposit);
        sediment -= deposit;

        break;
    }

    float slope = (currentHeight - lowestHeight);

    float erodeAmount = m_erosionRate * m_erosionStrength * slope;

    if (currentHeight > 0)
        erodeAmount = std::min(erodeAmount, currentHeight);

    if (erodeAmount > 0)
    {
        terrain.setAt(x, y, currentHeight - erodeAmount);
        sediment += erodeAmount;
    }

    float capacity = m_capacity * water;
    if (sediment > capacity)
    {
        float deposit = (sediment - capacity) * m_depositionRate;
        float newHeight = terrain.at(x,y) + deposit;
        terrain.setAt(x, y, newHeight);
        sediment -= deposit;
    }

    x = nx;
    y = ny;

```

```

        water -= (water * m_evaporation);
        if (water <= 0.00001f)
        {
            if (sediment > 0.0f)
            {
                float oldH = terrain.at(x,y);
                terrain.setAt(x, y, oldH + sediment);
                sediment = 0.0f;
            }
            break;
        }
    }
}

```

3.4 Интерфейс программного обеспечения

При запуске программы открывается окно, представленное на рисунке 3.2.

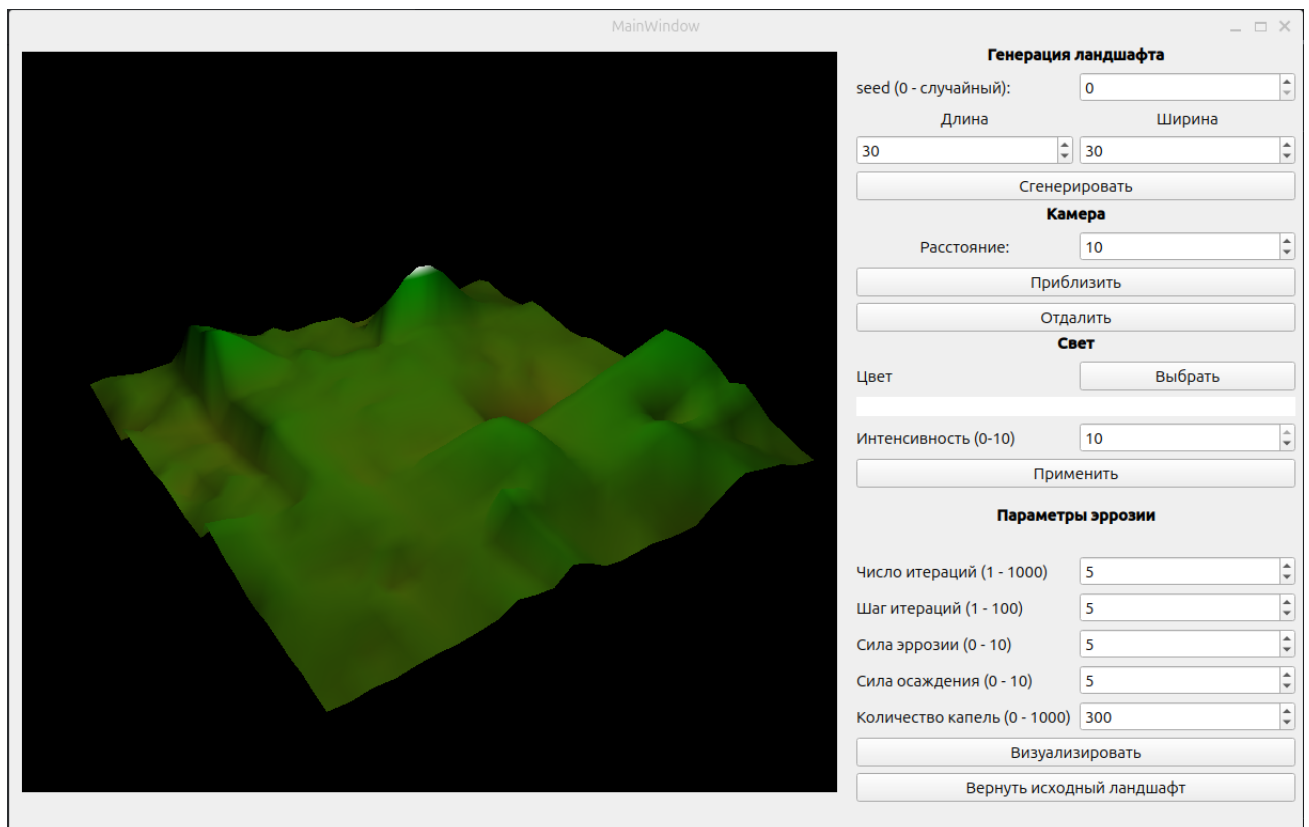


Рисунок 3.2 — Окно программы

Далее приведена информация о модулях интерфейса, с которыми пользователь может взаимодействовать.

Модуль изображения

Пользователь может вращать камеру вокруг ландшафта при помощи клавиш A/D клавиатуры – по горизонтали и W/S – по вертикали.

Модуль генерации

Пользователь может задать зерно генерации, а также длину и ширину итогового ландшафта. Нажатие кнопки «Сгенерировать» создает ландшафт по введенному зерну генерации полученный ландшафт визуализируется в модуле изображения.

Модуль камеры

Нажатие кнопок «Приблизить» и «Отдалить» уменьшает и увеличивает расстояние до центра Ландшафта соответственно.

Модуль источника цвета

Нажатие кнопки «Выбрать» открывает диалоговое окно с выбором цвета. После выбора цвета освещение сцены меняется. Кнопка «Применить» изменяет значение интенсивности в соответствии с введенным значением.

Модуль эрозии

Интерфейс позволяет задать следующие параметры эрозии:

- число итераций;
- шаг итераций;
- сила эрозии;
- сила осадения осадка;
- количество капель.

При нажатии на кнопку «Визуализировать» запускает процесс эрозии на введенное число итераций. Изображение обновляется в соответствии с введенным значением шага.

На рисунке 3.3 представлено изображение рельефа до визуализации водной эрозии.

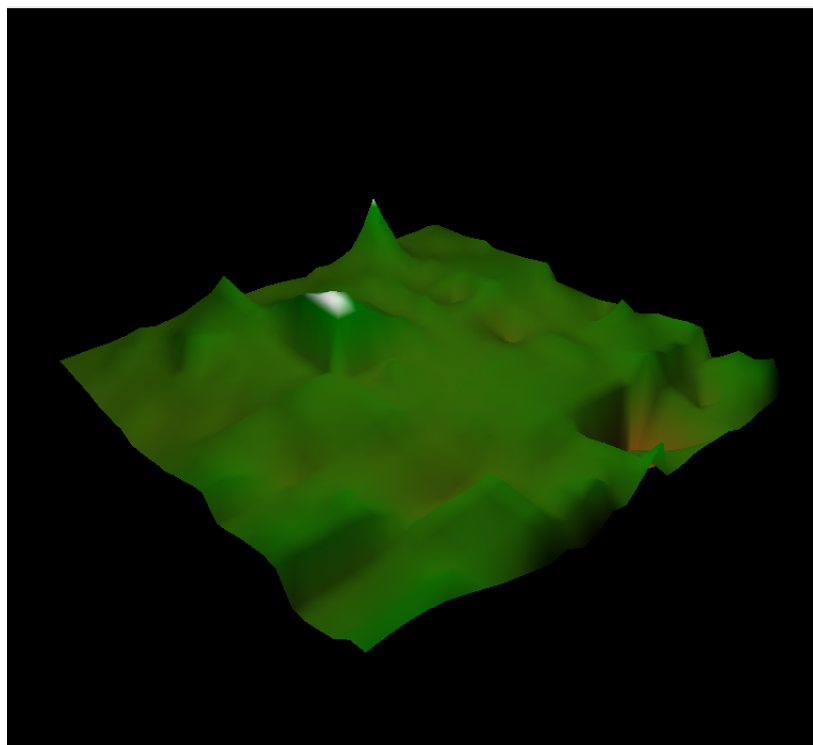


Рисунок 3.3 — Ландшафт до визуализации эрозии

На рисунке 3.4 представлено изображение рельефа после 50 итераций водной эрозии.

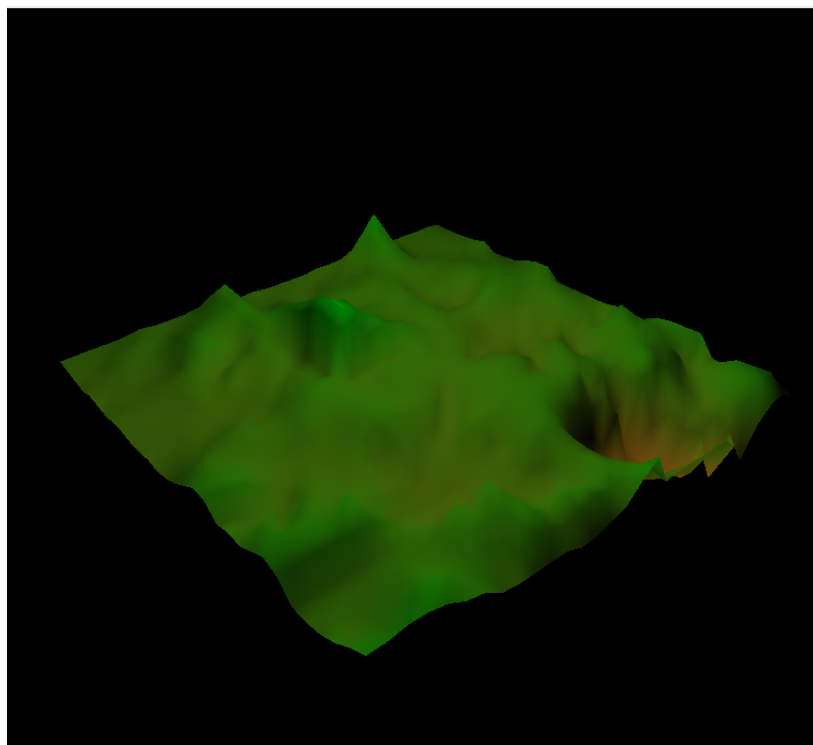


Рисунок 3.4 — Ландшафт после 50 итераций эрозии

На рисунке 3.5 представлено изображение рельефа после 100 итераций

водной эрозии.

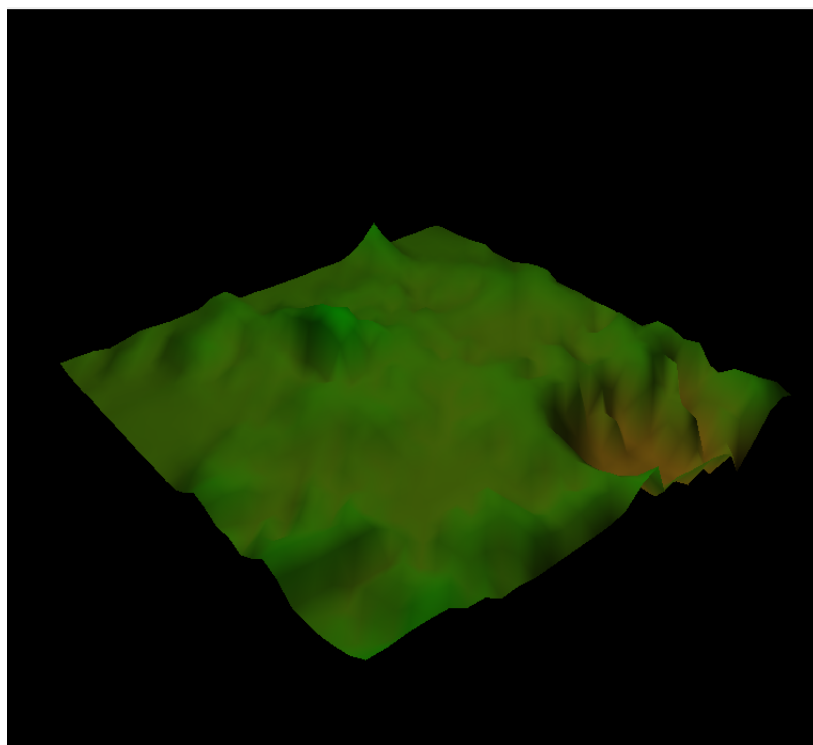


Рисунок 3.5 — Ландшафт после 100 итераций эрозии

На рисунке 3.6 представлено изображение рельефа после 200 итераций водной эрозии.

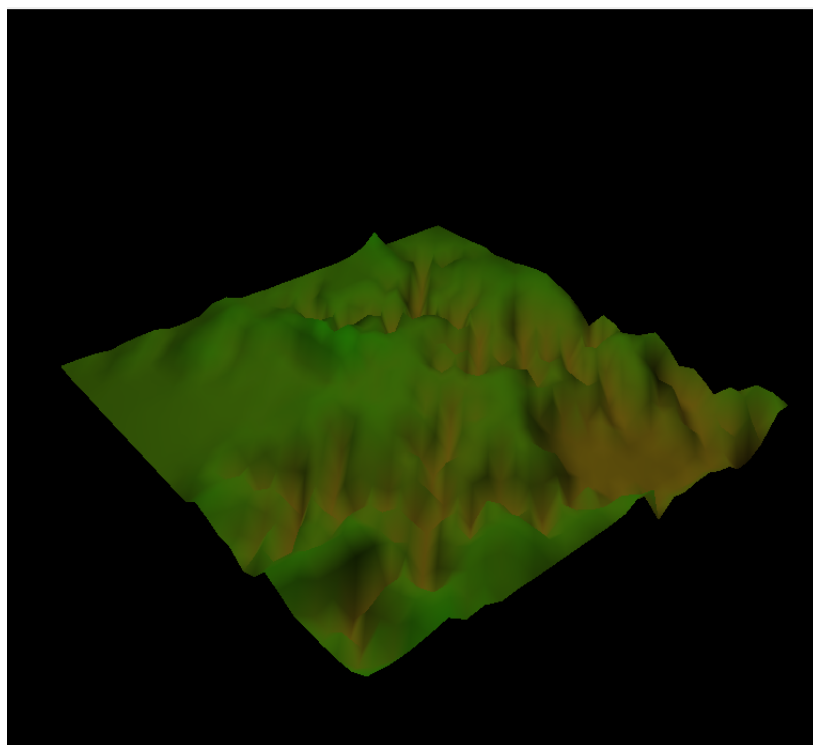


Рисунок 3.6 — Ландшафт после 200 итераций эрозии

Вывод

В этой части были приведены средства реализации. Также было реализовано ПО, выполняющее визуализацию водной эрозии рельефа, пример работы которого представлен в этой части.

4 Исследовательская часть

В этой части представлены технические характеристики ЭВМ, а также результаты исследования зависимости времени построения изображения ландшафта от размера карты высот.

4.1 Технические характеристики ЭВМ

Все замеры проводились на ЭВМ, характеристики которой приведены ниже:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16, 3.8 ГГц [12];
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS [13];
- оперативная память: 16,0 ГБ;
- число логических ядер – 16.

4.2 Результаты исследования

В ходе эксперимента производилось построение квадратных карт высот различных размеров, построение полигональной модели по построенной карте. Далее измерялось время рендеринга полученной полигональной сетки. Время замерялось с помощью функции *std :: chrono :: high_resolution_clock :: now* [14] из стандартной библиотеки *< chrono >* [15]. Для каждой размерности было произведено 100 замеров. Время отрисовки ландшафта не учитывалось. Полученные результаты, после нахождения среднего значения, представлены в таблице 4.1.

Таблица 4.1 — Среднее время рендеринга сцены в зависимости от размера высотной карты.

Размерность карты высот	Среднее время рендеринга (мс)
30	1.13
60	19.77
90	45.22
120	97.95
150	123.46
180	155.18
210	183.96
240	218.67
270	254.64
300	298.73

На рисунке 4.1 представлена визуализация результатов. График построен при помощи библиотеки *Python matplotlib* [16].

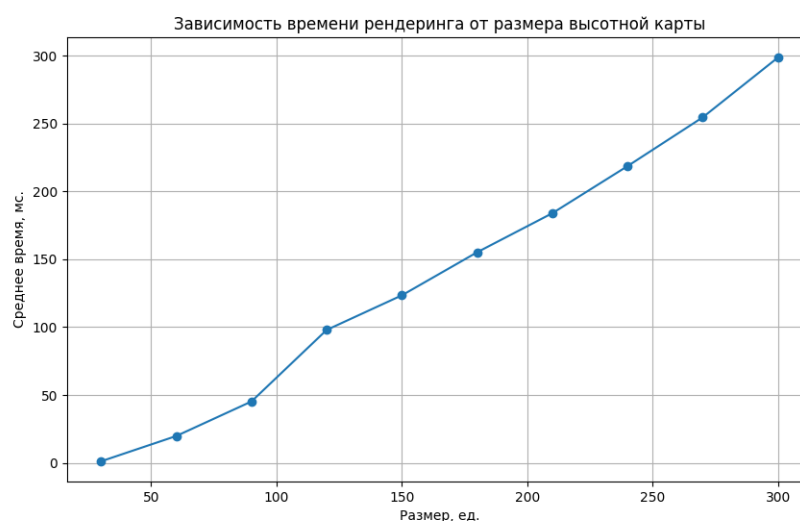


Рисунок 4.1 — Зависимость времени рендеринга от размера высотной карты

Из графика можно сделать вывод, что зависимость времени построения изображения ландшафта от размера квадратной карты высот линейна.

Вывод

В данном разделе описаны технические характеристики ЭВМ, на которой проводился эксперимент, а также представлены ход выполнения эксперимента и его результаты. Полученные данные подтвердили ожидания: время построения

изображения ландшафта линейно зависит от размера квадратной карты высот, что соответствует теоретической сложности алгоритма с использованием Z-буфера.

ЗАКЛЮЧЕНИЕ

Цель данной работы была достигнута: создано программное обеспечение для визуализации процесса водной эрозии рельефа с возможностью изменения физических и визуальных параметров модели. Все поставленные задачи были успешно решены:

- проведен анализ алгоритмов компьютерной графики для построения изображения;
- выбраны подходящие алгоритмы для построения изображения рельефа и визуализации водной эрозии;
- спроектирована архитектура программного обеспечения;
- определены и обоснованы средства реализации программного обеспечения;
- выполнено исследование зависимости времени построения изображения от размера карты высот, показавшее линейный характер зависимости;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Роджерс Д. Алгоритмические основы машинной графики: пер. с англ. СПб: БХВ-Петербург, 1989. — С. 512.
2. Демин А. Ю. Основы компьютерной графики. Издательство Томского политехнического университета, 2011. — С. 191.
3. Е.В. Шикин А.В. Боресков. Компьютерная графика. Динамика, реалистичные изображения. Москва «Диалог МИФИ», 1996. — С. 288.
4. Пермитина Т. О. Компьютерная графика. Учебное пособие. Томский государственный университет систем управления и радиоэлектроники (ТУСУР), 2012. — С. 145.
5. Шекихачев Ю. А. Хажметова А. Л. ИССЛЕДОВАНИЕ МЕХАНИЗМА ВОДНОЙ ЭРОЗИИ ПОЧВ. Известия Кабардино-Балкарского ГАУ, 2020. — С. 7.
6. Standard C++ [Электронный ресурс]. Режим доступа: <https://isocpp.org/> (Дата обращения: 16.11.2024).
7. CLion [Электронный ресурс]. Режим доступа: <https://www.jetbrains.com/clion/> (Дата обращения: 24.11.2024).
8. CLion [Электронный ресурс]. Режим доступа: <https://www.jetbrains.com/> (Дата обращения: 24.11.2024).
9. Qt Creator Documentation [Электронный ресурс]. Режим доступа: <https://doc.qt.io/qtcreator/> (Дата обращения: 24.11.2024).
10. Qt Framework [Электронный ресурс]. Режим доступа: <https://www.qt.io/product/framework> (Дата обращения: 24.11.2024).
11. glm [Электронный ресурс]. Режим доступа: <https://openframeworks.cc/documentation/glm/> (Дата обращения: 12.12.2024).
12. AMD Ryzen™ 7–5800H [Электронный ресурс]. Режим доступа: <https://www.amd.com/en/support/downloads/drivers.html/>

- processors/ryzen/ryzen-5000-series/amd-ryzen-7-5800h.html (Дата обращения: 15.11.2024).
13. Ubuntu 24.04 [Электронный ресурс]. Режим доступа: <https://ubuntu.com/blog/tag/ubuntu-24-04-lts> (Дата обращения: 15.11.2024).
 14. cpp high_resolution_clock [Электронный ресурс]. Режим доступа: https://en.cppreference.com/w/cpp/chrono/high_resolution_clock/now (Дата обращения: 15.11.2024).
 15. chrono [Электронный ресурс]. Режим доступа: <https://cplusplus.com/reference/chrono> (Дата обращения: 15.11.2024).
 16. matplotlib [Электронный ресурс]. Режим доступа: <https://matplotlib.org/> (Дата обращения: 15.11.2024).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе

Презентация содержит 10 слайдов.